

PORTADA

Manual Oficial de Usuario — miClub Gestión

Sistema Integral de Gestión Operativa

Versión del documento: 1.0 — Capítulo 1 Inicio

Fecha de generación: 30/06/2026

Estado: Versión Estable

Logo: apps/web/public/logo/miClub - Logo trans.png

Actualización post-admin: 06/08/2026. El capítulo operativo vigente de Administración se incorpora al final de este documento. Los artefactos HTML/PDF deben regenerarse antes de distribución externa.

Última sincronización de estructura y referencias técnicas: 19/09/2026.

ÍNDICE

Capítulo 0 Presentación

Capítulo 1 Inicio

Capítulo 2 Economía Club

Capítulo 3 Espacio Fitness

Capítulo 4 Salón

Capítulo 5 Aula

Capítulo 6 Local 1

Capítulo 7 Cantina

Capítulo 8 CRM

Capítulo 9 Migración

Capítulo 10 Administración

Capítulo 11 Arquitectura Técnica

Apéndices

CAPÍTULO 1 — INICIO

Alcance auditado. La pantalla Inicio está implementada por `HomeModule`, `useHomeDashboard`, `HomeMetricCards`, `RecentMovements` y `SectorDistribution`. Consume endpoints raíz legacy compatibles: `/summary`, `/members`, `/debtors`, `/sync-status`, `/club-finance-summary` y `/sector-operational-summary`. En el estado productivo declarado, PostgreSQL es la fuente oficial; Google Sheets fue retirado y sólo se conserva como antecedente histórico no ejecutable.

1. Panel Operativo

Qué representa: cabecera ejecutiva del Inicio con título “Panel operativo de miClub”, estado de sincronización, fecha de última sincronización y botón Sincronizar.

Estado: el frontend muestra “Con advertencias” si `/sync-status` devuelve error; muestra “Google Sheets conectado” si la fuente fuese `google_sheets`; para cualquier otro caso muestra “Datos mock/locales”. Inconsistencia: para `source=postgres` no existe etiqueta específica y el frontend cae en “Datos mock/locales”, aunque el backend productivo responde `source=postgres`. Impacto: puede confundir al usuario sobre el origen oficial. Solución: agregar etiqueta “PostgreSQL conectado”.

Última sincronización: se formatea en es-AR desde `lastSyncAt`. En PostgreSQL, `/sync-status` ejecuta health check; si es correcto actualiza `lastSyncAt` con la fecha/hora actual.

Botón Sincronizar: vuelve a ejecutar `loadHome` y recarga en paralelo todos los endpoints del Inicio. No importa datos ni escribe en PostgreSQL; solo refresca la vista.

Origen de datos: `DATA_SOURCE=postgres` activa PostgreSQL mediante `shouldUsePostgresDataSource`. Google Sheets no es backend ni proceso de migración disponible: su API, credenciales y dependencia ``googleapis`` fueron retiradas.

Plan y migración durante el onboarding

El asistente comienza en ``FREE``, que no permite importar. Se puede continuar gratis o elegir ``SOCIAL``, ``COMPLEX`` o ``CLUB``, cuyos entitlements habilitan el módulo **Migración** para usarlo

desde el panel después de finalizar. La guía XLSX está dividida en secciones desplegables. En esta fase no hay precios, cobro ni formulario de tarjeta; los planes pagos sólo se eligen libremente en pruebas y en producción esperan la integración de billing.

Tablas/servicios: /sync-status usa getPostgresHealth y validatePostgresEnv. Los datos funcionales se construyen desde people, enrollments, activities, sectors, movements, operational_balances y sheet_metric_snapshots.

2. Resumen Financiero

Endpoint: GET /club-finance-summary. Servicio: getPostgresClubFinanceSummary. Consultas principales: miclub.v_dashboard_basic, miclub.operational_balances, miclub.v_sector_settlement_balances y miclub.v_movements_enriched.

Liquidez: valor destacado. En la vista SQL se calcula como capital sin dólares + saldos positivos a liquidar + dólares valorizados por tipo de cambio del último operational_balances. En el servicio se toma de operational_balances más reciente o de v_dashboard_basic. Interpretación: capacidad económica operativa actual del club.

Caja: suma neta de movimientos administrativos completados con payment_method efectivo: ingresos/capital suman, egresos restan.

Banco: suma neta de movimientos administrativos completados con payment_method transferencia: ingresos/capital suman, egresos restan.

Dólares: suma neta de movimientos administrativos completados cuya categoría normalizada es DOLARES: ingresos/capital suman y egresos restan. Se presenta como USD entero redondeado.

Movimientos incluidos: movimientos de Administración completados para saldos reales; capital, ingresos y egresos según tipo; método de pago efectivo/transferencia; categoría Dólares para USD.

Movimientos excluidos: pendientes, cancelados/no completados para caja/banco/dólares; movimientos fuera de Administración para saldos administrativos; importes sin estado o categoría que no cumplen las reglas.

Ajustes manuales: operational_balances se ordena por cutoff_date y created_at descendente; el registro más reciente prevalece como foto operativa, incluyendo source_payload con tipo de cambio histórico.

3. Saldos Operativos

Endpoint: GET /club-finance-summary. Servicio: getPostgresClubFinanceSummary. Contrato compartido: ClubOperationsSummary.

Cuotas a Cobrar: suma de receivable_fee de inscripciones cuyo estado efectivo es adeudando. La cuota se normaliza si llegó inflada por mil: valores enteros $\geq 1.000.000$ divisibles por 1.000 se dividen por 1.000. Se excluyen cuotas ≤ 0 , abandonados y cancelados. Comisión aplicada: Fitness 50%; Salón 0%; Aula usa club_commission_percent acotado entre 0 y 1; otros sectores 0.

Saldos Pendientes: $\text{pendingNetBalance} = \text{pendingIncome} - \text{pendingExpenses}$. El SQL de la vista suma además futureReceivableFeesUntilMonthEnd, pero el servicio usa un fallback directo sobre v_movements_enriched filtrando source_payload.sheet = ADMINISTRACIÓN y estado operativo PENDIENTE o estado financiero pendiente; por eso el valor de la pantalla es neto de ingresos pendientes menos egresos pendientes administrativos.

Saldos a Liquidar: suma de saldos positivos a liquidar por sector. El servicio toma v_sector_settlement_balances con settlement_balance > 0 y complementa snapshots históricos fitness/salon/aula/local1. Se muestra con signo negativo porque representa obligación.

Saldo Proyectado: fórmula crítica del backend: Liquidez + Cuotas a cobrar + Saldos pendientes - Saldos a pagar. Cambia al modificar movimientos pendientes, pagos/estados de inscripciones, saldos de liquidación o liquidez.

Registros que afectan: enrollments.fee_amount/status/due_date/activity_id;
activities.club_commission_percent/sector_id; sectors.code/name;
movements.amount/type/status/financiamiento_status/source_payload; operational_balances;
sheet_metric_snapshots.

4. Ingresos por Sector

Endpoint /club-finance-summary. SQL: v_movements_enriched filtrada por movement_type=INGRESOS y operational_status=COMPLETADO, agrupada por sector_name con "Sin datos" como fallback. Se ordena por monto descendente y nombre ascendente, limitado a 4 filas. La pantalla marca con estrella el sector principal. No cuenta egresos, pendientes ni movimientos no completados.

5. Egresos por Sector

Mismo endpoint y servicio. Filtra movement_type=EGRESOS y operational_status=COMPLETADO, agrupa por sector_name, ordena por monto descendente y limita a 4. El primer egreso se marca con

señal visual negativa. No incluye ingresos ni pendientes.

6. Inscriptos

Endpoints: /summary, /members y /debtors. Servicios: getPostgresSummary, getPostgresMembers, getPostgresDebtors.

Total: cantidad de filas actuales de v_current_enrollments mapeadas como miembros.

Activos: inscripciones cuyo estado normalizado no es abandonado ni cancelado.

Al día: estado normalizado al_día. Nuevo inscripto con due_date vigente puede convertirse en “Al día” durante el mapeo.

Adeudando: estado normalizado adeudando. Nuevo inscripto vencido puede convertirse en “Adeudando”.

Abandonados: estado abandonado.

Cancelados: estado cancelado.

Cuota promedio: promedio simple de cuotas positivas de miembros activos cargados en frontend.

Adeudados por actividad: se filtran miembros deudores, se agrupan por actividad o modalidad, se ordenan por cantidad descendente y nombre; se muestran las primeras 3 y un contador de actividades restantes.

7. Inscriptos por actividad

Se calcula en frontend con miembros activos: excluye abandonados y cancelados, agrupa por actividad/modalidad, ordena por cantidad descendente y nombre, muestra las primeras 6 y destaca la primera. No usa un endpoint específico adicional: depende de /members.

8. Distribución Operativa por Sector

Endpoint: /sector-operational-summary. Servicio: getPostgresSectorOperationalSummary.

Espacio Fitness: total de inscriptos, activos, adeudados, monto adeudado, rentabilidad total,

rentabilidad mensual y saldo a liquidar. Rentabilidad y saldo provienen de `sheet_metric_snapshots`; si faltan, se marca pendiente de cálculo. Adeudados suman cuotas de miembros Fitness con estado adeudando.

Local 1: rentabilidad total, rentabilidad mensual, saldo a liquidar, total ventas, ventas últimos 30 días e ingreso destacado. Ventas relevantes son ingresos completados del sector Local 1 con categoría comisión o ventas. Ingreso destacado es el mayor monto y, ante empate operativo, el más reciente.

Salón: total, activos, rentabilidad total/mensual, actividad más popular y menos popular. Popularidad usa inscripciones no abandonadas/no canceladas agrupadas por actividad, ordenadas por cantidad.

Aula: total, activos, rentabilidad total/mensual, comisión promedio y actividad más popular. Comisión promedio proviene de `snapshot aula.average_commission`.

Cantina: kiosco, bebidas, CMV y rentabilidad total. Fórmula: $\text{ingresos kiosco} + \text{ingresos bebidas} - \text{egresos bebidas}$. Puede usar snapshots y fallback consultado.

CRM: total de miembros, activos, deudores y monto adeudado general. Es una tarjeta de visión transversal de cobranzas.

Interpretación Operativa

Primero mirar Liquidez y Saldo proyectado: si ambos bajan, existe riesgo financiero. Luego revisar Saldos a Liquidar para entender obligaciones próximas. Después analizar Cuotas a cobrar y Adeudados por actividad para detectar morosidad concentrada. Ingresos y egresos por sector muestran dónde se genera o consume dinero. Las tarjetas sectoriales permiten identificar qué unidad crece, cuál pierde rentabilidad y dónde conviene accionar. Un crecimiento sano combina más activos, ingresos completados y rentabilidad mensual positiva. Una alerta de morosidad aparece cuando aumentan Adeudando, Monto adeudado o Cuotas a cobrar sin mejora en liquidez. Una caída de ingresos se ve cuando el ranking de ingresos baja o cuando rentabilidad mensual queda pendiente, negativa o menor al patrón esperado.

Buenas prácticas

Actualizar Inicio antes de tomar decisiones. Leer saldos proyectados como estimación operativa, no como extracto bancario. Validar pagos pendientes antes de liquidar sectores. Revisar actividades con deudores altos. Usar tarjetas sectoriales para priorizar seguimiento.

Errores comunes

Confundir Saldos a Liquidar con dinero disponible. Leer Cuotas a cobrar como efectivo confirmado. Comparar Dólares con pesos sin tipo de cambio. Tratar Google Sheets como backend actual. Ignorar métricas “pendiente de cálculo”.

Recomendaciones

Agregar etiqueta PostgreSQL conectado. Revisar diferencias entre `v_dashboard_basic` y fallback del servicio en Saldos Pendientes. Consolidar snapshots críticos o reemplazarlos por vistas PostgreSQL autoritativas. Mantener pruebas sobre saldo proyectado, receivables y estados.

Inconsistencias detectadas

1. Etiqueta de origen en frontend: `source=postgres` se muestra como Datos mock/locales. Impacto alto de UX; solución: mapear postgres a PostgreSQL conectado.
2. Saldos Pendientes: `v_dashboard_basic` incluye cuotas futuras del mes en `pending_net_balance`, pero `getPostgresClubFinanceSummary` prioriza fallback directo sin cuotas futuras. Impacto: posible diferencia entre endpoint `/api/dashboard/basic` y pantalla Inicio. Solución: unificar fórmula.
3. Métricas sectoriales: varias rentabilidades y saldos dependen de `sheet_metric_snapshots` importados históricamente. Impacto: cobertura parcial si falta snapshot. Solución: vistas PostgreSQL completas por sector.
4. Rutas legacy raíz: el frontend consume `/summary`, `/members`, `/debtors` y no rutas `/api`. Impacto: contrato heredado; solución: documentar y migrar con compatibilidad.

Economía Club · gráficos analíticos anuales

La fila analítica anual de Economía Club utiliza `GET /api/economy/yearly-breakdown` con fecha de referencia opcional `?asOf=AAAA-MM-DD` y muestra los gráficos “Ingresos Operativos por Categoría” y “Gastos por Tipo” con una ventana móvil interanual inclusiva en zona ``America/Argentina/Buenos_Aires``: desde el mismo mes del año anterior hasta el mes actual, ambos extremos incluidos. Por esta regla aprobada se devuelven 13 puntos mensuales, con labels que incluyen mes y año, claves ``AAAA-MM`` para no mezclar meses iguales de años distintos y valores cero para meses sin movimientos. El backend filtra movimientos consolidados con ``operational_status = COMPLETADO``, usa ``movement_date >= fromMonth`` y ``movement_date < toExclusive``, excluye pendientes/cancelados/anulados por estado operativo y no modifica datos

históricos.

Categorías operativas para ingresos y gastos operativos: INSCRIPCIÓN, CUOTA, TURNOS, COMISIÓN, ALQUILER, EVENTOS, VENTAS, CLASES, CURSOS, KIOSCO y BEBIDAS. El gráfico de ingresos operativos incluye solo ``movement_type = INGRESOS``, excluye CAPITAL y publica únicamente categorías con total interanual mayor que cero.

El clasificador único de gastos aplica prioridad DEBT, SERVICES, TAXES, OPERATING, NON_OPERATING y UNCLASSIFIED. Las categorías no operativas son PUBLICIDAD, SALARIOS, MANTENIM., DEPÓSITOS, EXTRACCIONES, DÓLARES, REPARACIONES, VIÁTICOS, GANANCIA, PÉRDIDA, CMV, SEGUROS, LIMPIEZA, LIBRERÍA y OTROS. Deudas/Pasivos usa DEUDA y DEUDAS; Servicios usa LUZ, AGUA e INTERNET; Impuestos usa IMPUESTO e IMPUESTOS.

Convención de signos del gráfico “Gastos por Tipo”: para Gastos Operativos y Gastos No Operativos se suman únicamente EGRESOS; para Deudas/Pasivos, Servicios e Impuestos se calcula EGRESOS - INGRESOS y se preserva el signo sin ``ABS()``. Un valor negativo representa ingreso/reintegro neto superior al egreso del mes. Los movimientos no clasificados se reportan en metadata y en el script ``npm run audit:economy-yearly-breakdown -- --asOf=AAAA-MM-DD`` sin reclasificarlos como OTROS.

CAPÍTULO 9 — MIGRACIÓN XLSX (VIGENTE)

La importación soportada recibe un archivo ``.xlsx`` mediante ``POST /api/migration``. Es independiente de Google Sheets: no consulta Google Sheets API, no requiere credenciales ``GOOGLE_*` y no utiliza la dependencia ``googleapis``. Se debe ejecutar primero el dry-run, revisar errores y conservar el hash y reporte del lote antes de persistir. La guía operativa vigente es ``docs/imports/xlsx-runbook.md``.

Los antiguos procedimientos que leían rangos de una planilla Google o archivaban inscripciones de ese importador están en ``docs/history/`` y no deben ejecutarse en instalaciones nuevas. XLSX continúa soportado aunque Google Sheets haya sido retirado.

CAPÍTULO 10 — ADMINISTRACIÓN (ACTUALIZACIÓN POST-ADMIN)

Acceso y navegación

Ingresar con una membresía activa que tenga ``administration.view`` y elegir **Administración** en la navegación. Si aparece “No autorizado”, solicitar el permiso a un administrador; cambiar la URL o enviar otro ``clubId`` no cambia el club de la sesión. El panel muestra estados de carga, vacío y error con opción de reintentar.

Lectura del panel

Las tarjetas superiores resumen inscripciones activas, capacidad, trabajadores y actividades, con comparaciones cuando existe historial suficiente. Revisar la fecha y el mensaje de disponibilidad antes de interpretar una variación. Los rankings y tendencias son indicadores operativos, no un cierre contable.

En **Sectores**, seleccionar una fila para consultar responsable, capacidad, actividades y movimientos relacionados. Los sectores de sistema están protegidos. En **Actividades**, seleccionar una fila para consultar configuración, inscriptos, movimientos asociados mediante `activity\_id` y auditoría. En **Trabajadores**, abrir la ficha para consultar relación laboral, acceso, permisos y actividades. Estas tres fichas son de sólo lectura.

Si Trabajadores muestra una advertencia de fuente legacy, salario y fecha de ingreso pueden no estar disponibles; no completar esos datos por inferencia. **Actualizar** vuelve a consultar PostgreSQL y no importa datos externos.

Movimientos e inscripciones

Cargar Movimiento sólo se habilita con `movements.create`. Completar fecha, ingreso/egreso, categoría, sector, actividad si corresponde, concepto, contraparte, importe, medio de pago y estado inicial. El formulario envía una clave de idempotencia para evitar duplicados ante reintentos. Confirmar el resultado antes de volver a cargarlo. Un movimiento conciliado o aplicado a un pago no puede editarse; editar requiere `movements.edit` y anular exige motivo y `movements.cancel`.

Cargar Inscripción requiere `enrollments.create`. Elegir una persona y una actividad activa que genere inscripciones, indicar cuota, estado y fechas. El sistema rechaza referencias de otro club y una inscripción activa duplicada. Las futuras operaciones de edición y cancelación usarán `enrollments.edit` y `enrollments.cancel`. Los permisos amplios `finance:write` y `club:manage` sólo se aceptan como transición hasta el 2026-11-06; las membresías nuevas deben recibir siempre los permisos granulares.

Tareas y solicitudes

En **Tareas**, crear título, descripción opcional y vencimiento; luego cambiar entre Pendiente, En curso, Completada o Cancelada, o archivar. Si otra sesión modificó la tarea, recargar antes de reintentar. La etiqueta Vencida se calcula para tareas pendientes cuya fecha ya pasó.

Las solicitudes se pueden consultar con `requests.view`. Aprobar o rechazar requiere el permiso correspondiente; una decisión ya tomada no se repite y los tipos sin handler seguro no se ejecutan. Registrar un motivo claro aun cuando sea opcional.

Acciones todavía no disponibles

Gestionar categorías, trabajadores, cuotas y socios desde las tarjetas rápidas todavía no abre un

flujo completo. Reservas y Membresías aparecen como ****Próximamente**** hasta definir modelo, disponibilidad, pagos y cancelaciones. No usar SQL manual para sustituir esas funciones.

Buenas prácticas y soporte

Actualizar el panel antes de decidir, no compartir sesiones, no modificar el tenant desde herramientas del navegador y conservar el `requestId` de cualquier error. Ante una falla de escritura, no repetir compulsivamente: verificar primero si la operación quedó registrada. El índice vigente para localizar las fuentes técnicas, procedimientos de rollback y evidencia exigida es ``docs/operations/pre-reset-readiness.md``; el manual no afirma el estado aplicado de un entorno.